Project Description

Credit Card Approval Prediction

Project Description

Credit Card Approval Prediction is a Machine Learning–based web application designed to automate the process of predicting whether a credit card application should be approved or rejected based on applicant information. The system analyzes various customer attributes such as age, income, employment status, credit history, loan details, and other financial parameters to provide an instant prediction using a trained machine learning model.

The application is developed using the Flask framework for the backend and HTML, CSS, and JavaScript for the frontend. The prediction model is trained using the Scikit-learn library and utilizes data preprocessing techniques such as feature scaling and model serialization through Joblib. The trained model is integrated into the Flask application, enabling users to enter applicant information through an interactive web interface and receive immediate prediction results.

The application is deployed on the Render cloud platform, allowing users to access the prediction system through any modern web browser without requiring local installation. The system provides a responsive, secure, and user-friendly interface while ensuring fast prediction performance and reliable results.

The Credit Card Approval Prediction system assists financial institutions in reducing manual effort, minimizing processing time, and supporting data-driven decision-making during the credit card approval process.

Scenarios

Scenario 1

A salaried employee with a stable monthly income, good credit history, and low existing liabilities submits a credit card application. After entering the required details into the web application, the machine learning model predicts the application as **Approved**, enabling faster processing for the applicant.

Scenario 2

A customer with irregular employment, high outstanding debts, and a poor credit score applies for a credit card. Based on the provided information, the system predicts the application as **Rejected**, helping financial institutions identify high-risk applicants quickly.

Scenario 3

A bank executive uses the application to evaluate multiple customer applications during working hours. Instead of manually verifying every record, the system instantly predicts approval status, reducing processing time and improving operational efficiency.

Technical Architecture

Description

The Credit Card Approval Prediction system follows a modular **three-tier architecture** consisting of the Presentation Layer, Application Layer, and Machine Learning Layer to ensure efficient data processing and prediction.

The **Presentation Layer** provides an interactive web interface developed using HTML, CSS, and JavaScript. Users enter applicant details through responsive forms and submit the application for prediction.

The **Application Layer** is implemented using the Flask framework. It receives user input, validates the entered information, performs necessary preprocessing, loads the trained machine learning model, and communicates with the prediction engine.

The **Machine Learning Layer** contains the trained Scikit-learn classification model along with preprocessing objects such as the scaler. The model processes the applicant information and predicts whether the credit card application should be Approved or Rejected.

The entire application is deployed on the **Render Cloud Platform**, enabling secure online access through a web browser while maintaining high availability and scalability.

(If your project does not use a database, you can keep the note "Database: Not Applicable" in place of the ER diagram.)

Pre-requisites

- Flask Framework Knowledge
- Python Programming
- Machine Learning Fundamentals
- Scikit-learn Library
- Pandas Library
- NumPy Library
- HTML, CSS, and JavaScript
- Joblib Model Serialization
- Git and GitHub
- Render Cloud Deployment

- Visual Studio Code
- Apache JMeter for Performance Testing

Project Workflow

Milestone 1 – Project Planning and Model Development

Activity 1.1

Collect and analyze the Credit Card Approval dataset.

Activity 1.2

Perform data preprocessing including handling missing values, encoding categorical attributes, and feature scaling.

Activity 1.3

Train multiple Machine Learning classification models and select the best-performing model based on prediction accuracy.

Activity 1.4

Save the trained model and preprocessing objects using Joblib for integration into the web application.

Milestone 2 – Backend Development

Activity 2.1

Develop the Flask backend application to load the trained machine learning model.

Activity 2.2

Implement prediction logic, user input validation, preprocessing, and response generation.

Milestone 3 – Application Development

Activity 3.1

Develop the main application logic inside **app.py**, including routing, model loading, prediction functionality, and error handling.

Milestone 4 – Frontend Development

Activity 4.1

Design a responsive user interface using HTML, CSS, and JavaScript.

Activity 4.2

Develop interactive forms to collect applicant information and display prediction results dynamically.

Milestone 5 – Deployment

Activity 5.1

Configure the project for cloud deployment by installing all required dependencies.

Activity 5.2

Deploy the Flask application on the **Render Cloud Platform** and verify successful online access.

Milestone 6 – Conclusion

Evaluate the overall performance of the Credit Card Approval Prediction system, perform testing using Apache JMeter, analyze prediction accuracy and response time, and identify future enhancements to improve system performance and usability.

Milestone 1: Project Planning and Machine Learning Model Development

In this milestone, the primary focus is on collecting the dataset, preprocessing the data, selecting the most suitable Machine Learning algorithm, and developing a predictive model for credit card approval. The objective is to build an accurate classification model capable of predicting whether a credit card application should be approved or rejected based on customer information. The trained model is then saved for integration with the Flask web application.

Activity 1.1: Dataset Collection and Analysis

Before building the prediction model, an appropriate dataset containing customer and financial information must be collected. The quality of the dataset directly affects the prediction accuracy of the Machine Learning model.

Step 1 – Collect the Dataset

Obtain a Credit Card Approval dataset containing applicant information such as age, gender, income, employment status, education, marital status, loan details, credit history, and target approval status.

Step 2 – Analyze the Dataset

Load the dataset using the Pandas library and examine:

- Number of rows and columns.
- Feature names.

- Data types.
- Missing values.
- Duplicate records.
- Target variable distribution.

Example:

```
import pandas as pd


df = pd.read_csv("credit_card_dataset.csv")


print(df.head())

print(df.info())

print(df.describe())
```

Step 3 – Check Missing Values

Identify missing or null values.

```
df.isnull().sum()
```

Replace missing values using suitable preprocessing techniques such as mean, median, or mode depending on the feature type.

Step 4 – Explore Feature Relationships

Visualize important features using graphs to understand relationships between applicant attributes and approval status.

Examples include:

- Income Distribution
- Age Distribution
- Credit History Analysis
- Correlation Matrix
- Approval Class Distribution

Activity 1.2: Data Preprocessing

After collecting the dataset, preprocessing is performed to prepare the data for Machine Learning.

Perform the following preprocessing tasks:

• Remove duplicate records.

• Handle missing values.

• Convert categorical attributes into numerical values using Label Encoding or One-Hot Encoding.

• Normalize or standardize numerical features using StandardScaler.

• Separate features (X) and target variable (y).

Example:

```
from sklearn.preprocessing import StandardScaler


scaler = StandardScaler()


X_scaled = scaler.fit_transform(X)
```

Proper preprocessing improves model performance and prediction accuracy.

Activity 1.3: Machine Learning Model Selection

Several Machine Learning classification algorithms are evaluated to determine the best-performing model.

The models considered include:

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier
- Support Vector Machine (SVM)
- K-Nearest Neighbors (KNN)

Each model is trained and evaluated using performance metrics such as:

- Accuracy
- Precision
- Recall
- F1-Score

The model with the highest accuracy and stable performance is selected for deployment.

Example:

```
from sklearn.ensemble import RandomForestClassifier


model = RandomForestClassifier()


model.fit(X_train, y_train)
```

Activity 1.4: Saving the Trained Model

After selecting the best-performing Machine Learning model, it is saved using the Joblib library for future predictions within the Flask application.

Save the trained model

```
import joblib


joblib.dump(model, "model.pkl")
```

Save the scaler

```
joblib.dump(scaler, "scaler.pkl")
```

These files are later loaded by the Flask application during runtime to generate prediction results without retraining the model each time.

Milestone 1 Outcome

At the end of this milestone:

- ✓ Dataset successfully collected and analyzed.
- ✓ Data cleaned and preprocessed.
- ✓ Machine Learning model trained and evaluated.
- ✓ Best-performing classification model selected.
- ✓ Trained model and scaler saved using Joblib.
- ✓ Model prepared for integration into the Flask web application.

Milestone 2: Backend Development

Milestone 2 focuses on developing the backend of the Credit Card Approval Prediction application using the Flask framework. The backend acts as the communication layer between the user interface and the trained Machine Learning model. It processes user input, validates the entered data, loads the trained model, performs prediction, and returns the result to the frontend.

Activity 2.1: Develop the Flask Backend

Define Routes in Flask

The Flask application contains two primary routes.

Home Route

The home route loads the main webpage where users enter applicant details.

```
@app.route("/")
def home():
    return render_template("index.html")
```

Prediction Route

The prediction route accepts applicant information, processes the data, and predicts whether the application should be approved or rejected.

```
@app.route("/predict", methods=["POST"])
def predict():
```

Process User Input

Create an HTML form where users enter details such as:

- Applicant Age
- Annual Income
- Employment Status
- Credit History
- Loan Amount
- Number of Dependents
- Education
- Marital Status
- Existing Loan Information
- Other required applicant details

The backend retrieves these values using Flask's request object.

Example:

```
age = request.form["Age"]
income = request.form["Income"]
employment = request.form["Employment"]
credit_history = request.form["Credit_History"]
```

Input Validation

Before prediction, validate all user inputs.

Validation includes:

- Check for empty fields.
- Ensure numeric values are valid.
- Verify income is greater than zero.
- Reject invalid or unexpected input values.
- Prevent missing applicant information.

If validation fails, display an appropriate error message.

Load the Trained Model

The backend loads the saved Machine Learning model only once when the application starts.

Example:

```
import joblib

model = joblib.load("model.pkl")
scaler = joblib.load("scaler.pkl")
```

Data Preprocessing

Convert the received applicant details into the format expected by the trained model.

Example:

```
input_data = [[
    age,
    income,
    employment,
```

credit_history

]]

Apply the scaler before prediction.

scaled_data = scaler.transform(input_data)

Generate Prediction

The processed data is passed to the Machine Learning model.

prediction = model.predict(scaled_data)

Display Prediction Result

If the prediction value is:

0

Display

Credit Card Application Rejected

If the prediction value is

1

Display

Credit Card Application Approved

The prediction result is returned to the frontend and displayed on the webpage.

Activity 2.2: Integrating Machine Learning with Flask

The backend integrates the trained Machine Learning model with the Flask application to provide real-time predictions.

The integration process includes:

- Loading the saved model.
- Receiving applicant information.
- Preprocessing the data.
- Predicting approval status.
- Returning prediction results.

This allows users to receive instant predictions without retraining the model.

Error Handling

The backend handles possible runtime errors gracefully.

Examples include:

- Invalid input values.
- Missing applicant information.
- Model loading errors.
- Prediction failures.
- Internal server exceptions.

Appropriate error messages are displayed to improve user experience.

Example:

```
try:
    prediction = model.predict(scaled_data)
except Exception as e:
    return str(e)
```

Flask Request Flow

The complete request flow is as follows:

User Opens Website

Enter Applicant Details

Submit Application Form

Flask Receives Request

Validate User Input

▼

Load Trained Model

|

▼

Preprocess Input Data

|

▼

Predict Approval Status

|

▼

Return Prediction Result

|

▼

Display "Approved" or "Rejected"

Milestone 2 Outcome

At the end of this milestone:

- ✓ Flask backend successfully developed.
- ✓ User input validation implemented.
- ✓ Machine Learning model integrated with Flask.
- ✓ Applicant data preprocessed correctly.
- ✓ Prediction generated successfully.
- ✓ Result displayed instantly on the web interface.
- ✓ Error handling implemented for invalid inputs.

Milestone 3: App.py Development

Milestone 3 focuses on developing the **app.py** file, which acts as the central component of the Credit Card Approval Prediction application. The app.py file is responsible for initializing the Flask application, loading the trained Machine Learning model, defining application routes, processing user input, generating prediction results, and communicating with the frontend.

Activity 3.1: Writing the Main Application Logic

Import Required Libraries

The first step is importing all the required libraries for the application.

```
from flask import Flask, render_template, request
import joblib
import numpy as np
```

These libraries provide support for:

- Flask Web Framework
- HTML Template Rendering
- User Input Processing
- Machine Learning Model Loading
- Numerical Data Processing

Initialize Flask Application

Create the Flask application object.

```
app = Flask(__name__)
```

This initializes the web application and prepares it to receive HTTP requests.

Load the Trained Machine Learning Model

The trained model and preprocessing scaler are loaded when the application starts.

```
model = joblib.load("model.pkl")
scaler = joblib.load("scaler.pkl")
```

Loading the model once improves prediction speed because the model does not need to be reloaded for every request.

Define the Home Route

The home page displays the applicant information form.

```
@app.route("/")
def home():
    return render_template("index.html")
```

When users open the application, Flask renders the **index.html** page containing the credit card application form.

Define the Prediction Route

The prediction route receives applicant information from the HTML form.

@app.route("/predict", methods=["POST"])

def predict():

This route performs the following operations:

- Receives applicant information
- Validates user input
- Preprocesses the data
- Loads the trained model
- Generates prediction
- Displays the result

Retrieve User Input

Applicant information is collected from the submitted HTML form.

Example:

age = float(request.form["Age"])

income = float(request.form["Income"])

employment = request.form["Employment"]

credit_history = float(request.form["Credit_History"])

Other applicant details such as marital status, education, loan amount, and dependents are retrieved in a similar manner.

Prepare Input Data

The collected values are converted into a format suitable for the Machine Learning model.

```
input_data = np.array([[age,
                        income,
                        credit_history]])
```

The input data is arranged in the same order used during model training.

Apply Feature Scaling

If the trained model uses standardized data, apply the saved scaler before prediction.

scaled_data = scaler.transform(input_data)

Feature scaling ensures consistency between training and prediction datasets.

Generate Prediction

Pass the processed input to the trained Machine Learning model.

prediction = model.predict(scaled_data)

The model predicts whether the credit card application should be approved or rejected.

Display Prediction Result

The prediction output is checked and converted into a user-friendly message.

```
if prediction[0] == 1:
    result = "Credit Card Approved"
else:
    result = "Credit Card Rejected"
```

The result is displayed on the webpage.

```
return render_template("index.html", prediction=result)
```

Error Handling

The application handles unexpected errors using exception handling.

```
try:
    prediction = model.predict(scaled_data)
except Exception as e:
    return render_template("index.html",
                           prediction="Prediction Error")
```

This prevents application crashes and improves reliability.

Application Workflow

The execution flow of **app.py** is illustrated below.

Start Flask Application

Load Machine Learning Model

Load Scaler

User Opens Home Page

Enter Applicant Details

Submit Form

Receive Input

Validate Data

Scale Features

Predict Result

Display Approval Status

End

Advantages of app.py

- Centralized application logic.
- Easy integration with Machine Learning models.
- Fast prediction response.
- Simple route management.
- Easy maintenance and debugging.
- Supports future enhancements.
- Provides secure request handling.

Milestone 3 Outcome

At the end of this milestone:

- ✓ Flask application initialized successfully.
- ✓ Machine Learning model loaded correctly.
- ✓ Scaler integrated successfully.
- ✓ User input processed efficiently.
- ✓ Prediction generated successfully.
- ✓ Results displayed dynamically.
- ✓ Error handling implemented.
- ✓ Application ready for frontend integration.

Summary

The **app.py** file serves as the backbone of the Credit Card Approval Prediction system. It connects the frontend with the trained Machine Learning model, ensuring seamless data processing, accurate predictions, and efficient communication between different components of the application.

Milestone 4: Frontend Development

The frontend of the Credit Card Approval Prediction system is responsible for providing a simple, interactive, and user-friendly interface through which applicants can enter their information and receive instant prediction results. The frontend is developed using **HTML, CSS, and JavaScript**, ensuring responsiveness, accessibility, and ease of use.

The main objective of the frontend is to collect applicant information accurately and display the prediction result clearly without requiring technical knowledge from the user.

Activity 4.1: Designing the User Interface

The homepage of the application is designed to provide an intuitive interface where users can enter the required applicant details.

The interface includes:

- Project Title
- Applicant Information Form
- Input Fields
- Predict Button
- Prediction Result Section
- Footer Information

The design is kept clean and responsive so that users can easily navigate the application on desktops, laptops, tablets, and mobile devices.

Home Page Components

The homepage consists of the following sections:

Header

The header displays the project title:

Credit Card Approval Prediction

A short description explains that the application predicts whether a credit card application is likely to be approved or rejected using Machine Learning.

Applicant Information Form

The form collects applicant information such as:

- Age
- Annual Income
- Employment Status
- Marital Status
- Education Level
- Credit History
- Loan Amount
- Number of Dependents
- Existing Financial Obligations

Each input field is properly labeled to help users enter accurate information.

Submit Button

A **Predict** button is provided to submit the applicant details.

When clicked:

- The entered data is sent to the Flask backend.
- The backend processes the information.
- The Machine Learning model generates the prediction.

Prediction Result Section

After processing the input data, the application displays the prediction result on the same page.

Possible outputs include:

- **Credit Card Approved**
- **Credit Card Rejected**

The result is displayed prominently to ensure clear visibility for the user.

Activity 4.2: HTML Development

HTML is used to structure the web page and create the applicant information form.

The HTML page contains:

- Heading

- Form
- Input Fields
- Labels
- Buttons
- Result Display Area

Example:

```
<form action="/predict" method="POST">

<input type="number" name="Age">

<input type="number" name="Income">

<input type="submit" value="Predict">

</form>
```

The HTML structure ensures that all required applicant details are collected before submission.

CSS Styling

CSS is used to improve the appearance of the web application.

The following styling features are implemented:

- Professional layout
- Responsive design
- Consistent typography
- Attractive color scheme
- Proper spacing and alignment
- Rounded buttons and input fields
- Hover effects
- Mobile-friendly interface

The CSS improves user experience while maintaining a clean and modern appearance.

JavaScript Functionality

JavaScript is used to enhance the interactivity of the application.

The implemented features include:

- Basic client-side validation
- Prevention of empty form submission
- Dynamic user interaction
- Improved form usability
- Responsive feedback during submission

These features improve overall usability and reduce invalid submissions.

Responsive Design

The application is designed to work efficiently on different screen sizes.

Supported devices include:

- Desktop Computers
- Laptops
- Tablets
- Smartphones

Responsive design ensures that all interface components adjust automatically according to screen resolution.

User Interface Workflow

The frontend workflow is illustrated below.

User Opens Website

|

▼

View Credit Card Application Form

|

▼

Enter Applicant Details

|

▼

Click Predict Button

|

▼

Data Sent to Flask Backend

|

▼

Receive Prediction Result

|

▼

Display Approved / Rejected Status

Advantages of the Frontend Design

The developed frontend provides several advantages:

- Easy to use.
- Simple navigation.
- Responsive across devices.
- Attractive interface.
- Fast interaction.
- Instant prediction display.
- Improved user experience.
- Easy integration with the Flask backend.

Milestone 4 Outcome

At the end of this milestone:

- ✓ Responsive user interface successfully developed.
- ✓ Applicant information form implemented.
- ✓ HTML pages integrated with the Flask backend.
- ✓ CSS styling completed.

- ✓ JavaScript validation implemented.
- ✓ Prediction results displayed dynamically.
- ✓ Application tested successfully across multiple devices.

Milestone 5: Deployment and Testing

The final stage of the project focuses on deploying the Credit Card Approval Prediction application to a cloud platform and performing functional and performance testing. Deployment ensures that the application is accessible online, while testing verifies the correctness, reliability, and efficiency of the system.

The application is deployed on the **Render Cloud Platform**, enabling users to access the prediction system from any web browser without requiring local installation.

Activity 5.1: Cloud Deployment

After successful development and testing, the application is deployed using the Render cloud platform.

The deployment process includes the following steps:

Step 1 – Upload Project to GitHub

The complete project source code is uploaded to a GitHub repository. The repository contains:

- Flask Application (app.py)
- HTML Templates
- CSS and JavaScript Files
- Trained Machine Learning Model (model.pkl)
- Scaler File (scaler.pkl)
- requirements.txt
- README.md

Step 2 – Connect GitHub Repository to Render

The GitHub repository is connected to the Render platform.

Render automatically detects the Flask application and installs all dependencies listed in the requirements.txt file.

Step 3 – Configure Deployment

The following deployment settings are configured:

- **Runtime Environment:** Python
- **Build Command:** pip install -r requirements.txt
- **Start Command:** python app.py *(or the command used in your deployment)*

These settings ensure that the application is deployed correctly and all required packages are installed.

Step 4 – Access the Live Application

After successful deployment, the application becomes accessible through the following URL:

Deployment URL:

https://credit-card-approval-prediction-bu87.onrender.com

Users can access the application from any device with an internet connection.

Activity 5.2: Functional Testing

Functional testing is performed to verify that all features of the application work as expected.

The following functionalities were tested:

Test Case	Expected Result	Status
Home page loads successfully	Home page displayed	Passed
Applicant details accepted	Input processed correctly	Passed
Prediction generated	Approval/Rejection displayed	Passed
Invalid input handled	Error message displayed	Passed
Responsive design	Works on different devices	Passed

The application successfully passed all functional test cases.

Performance Testing Using Apache JMeter

Apache JMeter is used to evaluate the application's performance under multiple user requests.

The testing process includes:

- Sending multiple HTTP requests.
- Measuring response time.
- Calculating throughput.

- Recording error percentage.
- Evaluating server performance.

JMeter Test Configuration

The following configuration is used during testing:

- **Number of Virtual Users:** 50
- **Loop Count:** 10
- **Total Requests:** 500
- **Request Type:** HTTP POST
- **Target URL:** Credit Card Approval Prediction Application

Performance Test Results

Parameter	Value
Total Samples	500
Average Response Time	5323 ms
Minimum Response Time	2100 ms
Maximum Response Time	11870 ms
Error Rate	0%
Throughput	6.3 Requests/Second

The results indicate that the application processes requests efficiently while maintaining a zero error rate.

Website Pages

The deployed application consists of the following pages:

Home Page

Displays the Credit Card Approval Prediction interface where users enter applicant information.

Prediction Result Page

Displays the prediction outcome after processing the applicant details.

Possible outputs:

- Credit Card Approved
- Credit Card Rejected

Error Page (if applicable)

Displays appropriate error messages for invalid inputs or unexpected application errors.

Advantages of Deployment

Deploying the application on Render provides several benefits:

- Online accessibility.
- No local installation required.
- Easy maintenance.
- Automatic deployment from GitHub.
- High availability.
- Cloud scalability.
- Secure hosting environment.

Milestone 5 Outcome

At the end of this milestone:

- ✓ Application successfully deployed on Render.
- ✓ Functional testing completed.
- ✓ Performance testing conducted using Apache JMeter.
- ✓ Zero error rate achieved during testing.
- ✓ Prediction system verified successfully.
- ✓ Live application made accessible to users.

Milestone 6: Project Completion and Conclusion

The final milestone focuses on evaluating the overall performance of the Credit Card Approval Prediction system and summarizing the achievements of the project. The application has been successfully designed, developed, tested, and deployed using modern web technologies and Machine Learning techniques.

The system enables users to predict whether a credit card application is likely to be approved or rejected based on applicant details. By automating the prediction process, the application reduces manual effort, improves efficiency, and supports faster decision-making.

Activity 6.1: Project Evaluation

The developed system was evaluated based on the following criteria:

Functionality

The application successfully accepts applicant information, processes the data, and generates accurate prediction results using the trained Machine Learning model.

Performance

The deployed application demonstrates stable performance with acceptable response time and zero error rate during Apache JMeter testing.

Usability

The user interface is simple, responsive, and easy to understand. Users can submit applicant information and receive prediction results with minimal effort.

Reliability

The Flask backend successfully integrates with the trained Machine Learning model, ensuring consistent prediction results and stable application performance

Project Achievements

The following objectives were successfully achieved during the project:

- Developed a Machine Learning model for credit card approval prediction.
- Integrated the trained model with a Flask web application.
- Designed a responsive frontend using HTML, CSS, and JavaScript.
- Implemented secure data processing and prediction logic.
- Successfully deployed the application on the Render cloud platform.
- Conducted functional and performance testing using Apache JMeter.
- Achieved reliable prediction performance with zero error rate during testing.

Future Enhancements

The Credit Card Approval Prediction system can be enhanced further by implementing additional features such as:

- User Authentication and Authorization.
- Database integration for storing applicant records.
- Email notifications for prediction results.
- Improved Machine Learning models for higher prediction accuracy.
- Real-time API integration with banking systems.
- Dashboard for administrators to monitor prediction statistics.
- Explainable AI (XAI) features to provide reasons for prediction results.
- Mobile application for Android and iOS platforms.
- Multi-language support for wider accessibility.
- Advanced analytics and reporting modules.

These enhancements will improve usability, scalability, and overall system performance.

Conclusion

The **Credit Card Approval Prediction** project successfully demonstrates the practical application of Machine Learning in the banking and financial sector. By combining a trained classification model with a Flask-based web application, the system provides accurate and efficient prediction of credit card application outcomes.

The developed application simplifies the credit card approval process by reducing manual effort, minimizing processing time, and supporting data-driven decision-making. The responsive web interface allows users to enter applicant information easily, while the integrated Machine Learning model generates prediction results instantly.

Deployment on the Render cloud platform ensures online accessibility, and performance testing using Apache JMeter confirms that the application operates reliably under multiple concurrent requests. Overall, the project meets its intended objectives and serves as a scalable foundation for future improvements in intelligent financial decision support systems.

References

The following resources were referred to during the development of this project:

1. Python Official Documentation.
2. Flask Official Documentation.
3. Scikit-learn Documentation.

4. Pandas Documentation.
5. NumPy Documentation.
6. Joblib Documentation.
7. Apache JMeter User Manual.
8. Render Cloud Deployment Documentation.
9. GitHub Documentation.
10. Research articles on Credit Card Approval Prediction using Machine Learning.

Software Used

Software	**Purpose**
Python	Backend Development
Flask	Web Framework
HTML	Web Page Structure
CSS	Styling
JavaScript	Client-side Interactivity
Scikit-learn	Machine Learning Model
Pandas	Data Processing
NumPy	Numerical Computation
Joblib	Model Serialization
Git	Version Control
GitHub	Source Code Management
Render	Cloud Deployment
Apache JMeter	Performance Testing

Hardware Requirements

Component	**Specification**
Processor	Intel Core i3 or Above

Component	Specification
RAM	Minimum 4 GB
Storage	500 MB Free Space
Operating System	Windows 10 / Windows 11
Internet	Required for Deployment

Final Outcome

The Credit Card Approval Prediction application was successfully:

- ✓ Planned
- ✓ Designed
- ✓ Developed
- ✓ Tested
- ✓ Deployed
- ✓ Documented

The project fulfills all the defined objectives and demonstrates the effective use of Machine Learning and Web Development technologies to solve a real-world financial prediction problem.